

PySpark DataFrame Practice Dataset and Transformation Questions

Sample Employee Dataset

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder.appName("PySparkPractice").getOrCreate()

# Employee Dataset
employee_data = [
    (101, "Sravan", "Data Engineer", "IT", 75000, "Hyderabad", 28,
     "2021-05-10", "Male"),
    (102, "Ravi", "Software Engineer", "IT", 68000, "Bangalore", 30,
     "2020-03-15", "Male"),
    (103, "Priya", "Data Analyst", "Analytics", 62000, "Chennai", 26,
     "2022-01-12", "Female"),
    (104, "Kiran", "Manager", "HR", 90000, "Mumbai", 35, "2018-07-19",
     "Male"),
    (105, "Anjali", "HR Executive", "HR", 45000, "Pune", 24, "2023-02-20",
     "Female"),
    (106, "Vikram", "Data Scientist", "Analytics", 98000, "Delhi", 32,
     "2019-11-25", "Male"),
    (107, "Sneha", "Developer", "IT", 71000, "Hyderabad", 27, "2021-08-17",
     "Female"),
    (108, "Rahul", "Tester", "QA", 55000, "Chennai", 29, "2020-06-10",
     "Male"),
    (109, "Meena", "QA Lead", "QA", 83000, "Bangalore", 33, "2017-09-14",
     "Female"),
    (110, "Arjun", "Support Engineer", "Support", 50000, "Pune", 31,
     "2022-04-11", "Male")
]

columns = ["emp_id", "name", "designation", "department", "salary", "city",
           "age", "joining_date", "gender"]

emp_df = spark.createDataFrame(employee_data, columns)

emp_df.show()
```

1. select() Transformation Questions

1. Select only employee name and salary columns.

2. Select emp_id, department and city columns.
 3. Select all employees with only name and age.
 4. Select designation and salary columns.
 5. Select emp_id, name and joining_date columns.
-

2. filter() / where() Transformation Questions

1. Find employees whose salary is greater than 70000.
 2. Find employees working in IT department.
 3. Find employees whose age is less than 30.
 4. Find female employees.
 5. Find employees from Hyderabad city.
-

3. withColumn() Transformation Questions

1. Add a new column bonus with 10% of salary.
 2. Add a new column tax with 5% deduction.
 3. Add a new column salary_increment with salary + 5000.
 4. Create a new column age_group based on age.
 5. Add a column yearly_salary = salary * 12.
-

4. withColumnRenamed() Questions

1. Rename emp_id to employee_id.
 2. Rename designation to role.
 3. Rename salary to monthly_salary.
 4. Rename city to work_location.
 5. Rename joining_date to doj.
-

5. drop() Transformation Questions

1. Drop age column.
 2. Drop gender column.
 3. Drop joining_date column.
 4. Drop city and age columns.
 5. Drop designation column.
-

6. distinct() Questions

1. Find distinct departments.

2. Find distinct cities.
 3. Find distinct designations.
 4. Find distinct genders.
 5. Find unique department and city combinations.
-

7. dropDuplicates() Questions

1. Remove duplicate rows based on emp_id.
 2. Remove duplicates based on department.
 3. Remove duplicates based on city.
 4. Remove duplicates based on department and city.
 5. Remove duplicate employees based on name.
-

8. sort() / orderBy() Questions

1. Sort employees by salary ascending.
 2. Sort employees by age descending.
 3. Sort employees by department and salary.
 4. Sort employees by city.
 5. Sort employees by joining_date descending.
-

9. groupBy() Questions

1. Find average salary by department.
 2. Find maximum salary in each department.
 3. Find minimum age in each department.
 4. Count employees in each city.
 5. Find total salary by gender.
-

10. agg() Questions

1. Find total salary of all employees.
 2. Find average employee age.
 3. Find max and min salary together.
 4. Find count of employees.
 5. Find average salary and average age department-wise.
-

11. join() Questions

Department Dataset

```
department_data = [  
    ("IT", "John"),  
    ("HR", "Smith"),  
    ("QA", "David"),  
    ("Analytics", "Kevin"),  
    ("Support", "Robert")  
]  
  
dept_columns = ["department", "manager"]  
  
dept_df = spark.createDataFrame(department_data, dept_columns)
```

1. Perform inner join between emp_df and dept_df.
 2. Perform left join.
 3. Perform right join.
 4. Perform full outer join.
 5. Find employees along with manager names.
-

12. union() Questions

Additional Dataset

```
new_emp_data = [  
    (111, "Teja", "Developer", "IT", 72000, "Hyderabad", 26, "2023-01-10",  
    "Male"),  
    (112, "Divya", "Analyst", "Analytics", 65000, "Bangalore", 25,  
    "2022-11-05", "Female")  
]  
  
new_emp_df = spark.createDataFrame(new_emp_data, columns)
```

1. Union emp_df with new_emp_df.
 2. Count total rows after union.
 3. Remove duplicates after union.
 4. Find all IT employees after union.
 5. Sort unioned dataframe by salary.
-

13. unionByName() Questions

1. Create another dataframe with shuffled columns and perform unionByName.
 2. Use allowMissingColumns=True.
 3. Add extra column and perform unionByName.
 4. Compare union vs unionByName.
 5. Merge employee datasets with different schemas.
-

14. limit() Questions

1. Show first 3 records.
 2. Show first 5 employees.
 3. Get top 2 highest salary employees.
 4. Get first 4 IT employees.
 5. Show first employee record.
-

15. sample() Questions

1. Take 50% sample of data.
 2. Take 30% sample with seed.
 3. Take sample from IT employees.
 4. Compare full data vs sampled data.
 5. Take random sample of 5 rows.
-

16. explode() Questions

Skills Dataset

```
skills_data = [  
    (101, ["Python", "Spark", "Azure"]),  
    (102, ["Java", "SQL"]),  
    (103, ["Power BI", "SQL"])  
]  
  
skills_columns = ["emp_id", "skills"]  
  
skills_df = spark.createDataFrame(skills_data, skills_columns)
```

1. Explode skills array into multiple rows.
2. Count total skills.
3. Find employees with Python skill.
4. Find distinct skills.

5. Group employees by skill.
-

17. split() Questions

1. Split joining_date into year, month and day.
 2. Split designation into words.
 3. Split city names.
 4. Extract year from joining_date.
 5. Create separate columns using split.
-

18. concat() / concat_ws() Questions

1. Concatenate name and department.
 2. Create full employee details column.
 3. Concatenate city and department with hyphen.
 4. Create employee label using concat_ws.
 5. Combine name and designation.
-

19. cast() Questions

1. Cast salary to double.
 2. Cast age to string.
 3. Convert joining_date to date type.
 4. Cast emp_id to string.
 5. Create numeric bonus column.
-

20. alias() Questions

1. Display salary as monthly_salary.
 2. Display department as dept_name.
 3. Use alias in aggregation.
 4. Rename average salary column using alias.
 5. Use alias in joins.
-

21. lit() Questions

1. Add company_name column.
2. Add country column with value India.
3. Add constant bonus column.
4. Add status column as Active.

5. Add training column with Yes.
-

22. when() / otherwise() Questions

1. Categorize salary as High and Low.
 2. Categorize employees based on age.
 3. Create experience level column.
 4. Mark employees eligible for bonus.
 5. Create department category column.
-

23. substring() Questions

1. Extract first 3 characters from name.
 2. Extract year from joining_date.
 3. Extract first 2 letters of department.
 4. Extract last 3 characters from designation.
 5. Create short employee code.
-

24. regexp_replace() Questions

1. Replace spaces in designation with underscore.
 2. Remove vowels from names.
 3. Replace Hyderabad with HYD.
 4. Remove special characters.
 5. Standardize department names.
-

25. like() Questions

1. Find names starting with S.
 2. Find designations ending with Engineer.
 3. Find cities containing 'a'.
 4. Find departments starting with A.
 5. Find employees whose name contains 'ra'.
-

26. isin() Questions

1. Find employees from Hyderabad and Bangalore.
2. Find employees in IT and QA departments.
3. Find employees with age 28, 30 and 35.
4. Filter female employees from Pune and Chennai.

5. Find employees with specific emp_ids.
-

27. between() Questions

1. Find employees with salary between 50000 and 80000.
 2. Find employees aged between 25 and 30.
 3. Find salaries between 60000 and 90000.
 4. Find employees joined between years.
 5. Find employees with emp_id between 102 and 108.
-

28. pivot() Questions

1. Pivot department with average salary.
 2. Pivot city with employee count.
 3. Pivot gender with total salary.
 4. Pivot department with maximum age.
 5. Create department-wise summary table.
-

29. unpivot / stack Questions

1. Convert pivoted salary table back to rows.
 2. Unpivot city data.
 3. Convert department columns into rows.
 4. Practice stack function.
 5. Create normalized dataframe.
-

30. Window Functions Questions

1. Find rank of employees based on salary.
 2. Find dense_rank department-wise.
 3. Find row_number for employees.
 4. Find lead salary.
 5. Find lag salary.
-

31. repartition() Questions

1. Repartition dataframe into 4 partitions.
2. Repartition by department.
3. Check partition count.
4. Compare repartition and coalesce.

5. Repartition large datasets.
-

32. coalesce() Questions

1. Reduce partitions to 2.
 2. Check performance difference.
 3. Use coalesce before writing files.
 4. Compare shuffle behavior.
 5. Optimize output files.
-

33. cache() Questions

1. Cache employee dataframe.
 2. Perform multiple actions after cache.
 3. Compare execution time.
 4. Use cache with aggregation.
 5. Unpersist cached dataframe.
-

34. fillna() Questions

1. Fill null salaries with 0.
 2. Fill null city with Unknown.
 3. Fill multiple columns.
 4. Fill null ages with average age.
 5. Replace null strings.
-

35. replace() Questions

1. Replace Hyderabad with HYD.
 2. Replace IT with Information Technology.
 3. Replace Male/Female with M/F.
 4. Replace QA with Testing.
 5. Replace multiple values together.
-

36. na.drop() Questions

1. Drop rows with null salary.
2. Drop rows with any null value.
3. Drop rows with all null values.
4. Drop rows based on threshold.

5. Drop rows with null city.
-

37. na.fill() Questions

1. Fill null age with 25.
 2. Fill null department with Unknown.
 3. Fill salary with average.
 4. Fill multiple columns together.
 5. Fill missing string values.
-

38. map() Transformation Questions (RDD)

1. Convert names to uppercase.
 2. Increase salary by 10%.
 3. Extract employee names.
 4. Convert department names to lowercase.
 5. Create employee summary strings.
-

39. flatMap() Questions (RDD)

1. Split names into characters.
 2. Split designation words.
 3. Extract all skills.
 4. Flatten nested lists.
 5. Generate words from sentences.
-

40. reduceByKey() Questions (RDD)

1. Count employees by department.
 2. Sum salaries department-wise.
 3. Count city-wise employees.
 4. Aggregate ages by department.
 5. Find maximum salary by city.
-

41. mapPartitions() Questions

1. Process data partition-wise.
2. Count rows per partition.
3. Convert names to uppercase partition-wise.
4. Optimize heavy computations.

5. Compare map vs mapPartitions.
-

42. zipWithIndex() Questions

1. Add index to each employee.
 2. Generate sequence numbers.
 3. Create row IDs.
 4. Compare monotonically_increasing_id.
 5. Generate indexed employee list.
-

43. crossJoin() Questions

1. Cross join employees with departments.
 2. Generate all combinations.
 3. Find total combinations count.
 4. Cross join city and department.
 5. Understand Cartesian product.
-

44. except() Questions

1. Find employees not in new_emp_df.
 2. Compare two dataframes.
 3. Remove matching rows.
 4. Find difference between datasets.
 5. Validate missing employees.
-

45. intersect() Questions

1. Find common employees.
 2. Compare datasets.
 3. Find common departments.
 4. Find common cities.
 5. Validate matching records.
-

46. cube() Questions

1. Perform cube on department and city.
2. Find multi-dimensional aggregates.
3. Analyze salary combinations.
4. Find total salary cube.

5. Compare rollup and cube.
-

47. rollup() Questions

1. Rollup department and city.
 2. Create hierarchical summaries.
 3. Find total salaries.
 4. Analyze rollup output.
 5. Compare cube and rollup.
-

48. broadcast() Questions

1. Broadcast join small department table.
 2. Compare join performance.
 3. Optimize joins.
 4. Use broadcast with lookup table.
 5. Analyze execution plan.
-

49. explode_outer() Questions

1. Explode arrays with null values.
 2. Compare explode and explode_outer.
 3. Handle empty arrays.
 4. Preserve null rows.
 5. Analyze output differences.
-

50. array_contains() Questions

1. Find employees with Spark skill.
 2. Filter Python developers.
 3. Search arrays.
 4. Find employees with SQL skill.
 5. Check Azure skill availability.
-

Advanced Mini Projects

Project 1: Employee Salary Analytics

- Find top 3 highest salaries department-wise.
- Find average salary city-wise.

- Categorize employees into salary bands.
- Generate yearly salary report.
- Find employees earning above department average.

Project 2: HR Dashboard

- Count male and female employees.
- Find average age department-wise.
- Generate hiring trends by year.
- Create city-wise employee distribution.
- Identify highest paid employees.

Project 3: Data Engineering Scenarios

- Remove duplicate records.
 - Handle missing data.
 - Join multiple datasets.
 - Optimize partitions.
 - Create aggregated summary tables.
-

Interview-Oriented Practice

1. Difference between map and flatMap.
 2. Difference between groupBy and reduceByKey.
 3. Difference between repartition and coalesce.
 4. Difference between cache and persist.
 5. Difference between union and unionByName.
 6. Difference between inner join and left join.
 7. Difference between distinct and dropDuplicates.
 8. Difference between rank and dense_rank.
 9. Difference between explode and explode_outer.
 10. Difference between rollup and cube.
-

Bonus Challenge Questions

1. Find second highest salary using window functions.
2. Find duplicate employee records.
3. Find employees joined in the same year.
4. Generate department-wise top earners.
5. Create running total salary.
6. Find cumulative average salary.
7. Identify employees with same salary.
8. Find longest employee name.
9. Create dynamic salary bands.
10. Build complete employee reporting pipeline.